

Singapore Institute of Technology
Information and Communications Technology Cluster

AAI4001 Capstone Project

Capstone Project Progress Report Submission Form (Form E1)

A.Preamble

This form duly filled and signed must be attached to each Capstone Project Progress Report submitted.

Reporting Period:

(Reporting Period): 13/10/2025 to 5/12/2025

Candidate Particulars:

(Name of Candidate): Donovan Leong Jia Le

(Organization): Panasonic Information Systems Company Asia Pacific

(Project Period): 13/10/2025 to 10/4/2026

Main Academic Supervisor Details:

(Name): Bogdan Cautis

(Email Address): bogdan.cautis@singaporetech.edu.sg

(Contact Number): 8229 0321

(Designation): Associate Professor

Industry Supervisor Details:

(Name): Praveen Reddy Satti

(Email Address): praveen.reddysatti@sg.panasonic.com

(Contact Number): 96501275

(Designation): SAP Application Delivery Management Lead, Application Managed Service

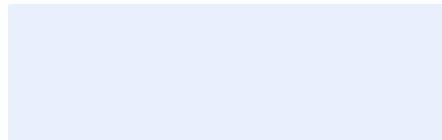
(Department / Division): Managed Services ADM/AMS, PISCAP

B. Declaration of Conformity

B.1 Declaration by Industry Supervisor

I hereby acknowledge that I have read and understood the contents of the Capstone Project Progress Report, and deem the contents appropriate for release to the Singapore Institute of Technology for use in the assessment of the candidate.

Signature



Name of Industry Supervisor: Praveen Reddy Satti
Designation: SAP Application Delivery Management Lead, Application Managed Service
Department: Managed Services ADM/AMS, PISCAP
Contact No: 96501275
Email: praveen.reddysatti@sg.panasonic.com
Date: 8/12/2025

B.2 Declaration by Candidate

I hereby acknowledge that I have engaged and discussed with my Industry Supervisor on the contents of the Capstone Project Progress Report, and have sought approval for the release of the report to the Singapore Institute of Technology.

Signature



Name of Candidate: Donovan Leong Jia Le
Contact No: 91913435
Email: 2302674@sit.singaporetech.edu.sg
Date: 7/12/2025

END OF FORM E



Information and Communications Technology Cluster

ITSM Incident Ticket Classification using Natural Language Processing

Capstone Project Progress Report (Panasonic Information Systems Company Asia Pacific)

For the reporting period from 13/10/2025 to 5/12/2025

Donovan Leong Jia Le
2302674

Name of Industry Supervisor: Praveen Reddy Satti

Name of Academic Supervisor: Bogdan Cautis

Submitted as part of the requirement for AAI4001 Capstone Project

A. PREAMBLE.....	1
B. DECLARATION OF CONFORMITY	2
<i>B.1 Declaration by Industry Supervisor</i>	<i>2</i>
<i>B.2 Declaration by Candidate.....</i>	<i>2</i>
1. INTRODUCTION.....	3
1.1 PROJECT BACKGROUND AND CONTEXT.....	3
1.2 PROBLEM STATEMENT	3
1.3 PROJECT OBJECTIVES AND SCOPE	4
2. LITERATURE REVIEW AND THEORETICAL FOUNDATIONS	5
2.1 EVOLUTION OF TEXT REPRESENTATION IN INFORMATION RETRIEVAL	5
2.2 TRANSFORMER ARCHITECTURES AND SELF-ATTENTION MECHANISMS	5
2.3 SENTENCE TRANSFORMERS AND CONTRASTIVE LEARNING	6
2.4 MODEL SELECTION AND COMPARATIVE FRAMEWORK.....	7
2.5 APPLICATIONS IN IT SERVICE MANAGEMENT.....	7
3. METHODOLOGY AND SYSTEM ARCHITECTURE.....	8
3.1 OVERALL SYSTEM ARCHITECTURE AND COMPONENT OVERVIEW	8
<i>Component 1: Data Pipeline Service (Status: Completed)</i>	<i>8</i>
<i>Component 2: Model Training Service (Status: In Progress)</i>	<i>8</i>
<i>Component 3: Model Inference Service (Status: Planned)</i>	<i>8</i>
<i>Component 4: Dashboard and Visualization Service (Status: In Progress)</i>	<i>9</i>
<i>Component 5: API Gateway Service (Status: Planned).....</i>	<i>9</i>
3.2 DATA ACQUISITION AND PROCESSING PIPELINE	10
<i>Data Pipeline Architecture.....</i>	<i>11</i>
3.3 MODEL TRAINING METHODOLOGY AND PRELIMINARY RESULTS.....	12
<i>Classification Task:</i>	<i>12</i>
<i>Metric Learning Task:.....</i>	<i>12</i>
<i>Preliminary MPnet Training Results:.....</i>	<i>13</i>
3.4 EVALUATION FRAMEWORK AND METRICS.....	15
4. IMPLEMENTATION PROGRESS AND RESULTS	15
4.1 COMPLETED: DATA INFRASTRUCTURE AND QUALITY ANALYSIS	15
4.2 COMPLETED: EVALUATION FRAMEWORK AND TESTING INFRASTRUCTURE.....	16
4.3 PROJECTED MODEL PERFORMANCE BASED ON LITERATURE	16
5. CHALLENGES, RISKS, AND MITIGATION STRATEGIES.....	17
5.1 COMPUTATIONAL RESOURCE CONSTRAINTS	17
5.2 CLASS IMBALANCE AND MINORITY CATEGORY PERFORMANCE.....	17
5.3 GROUND TRUTH LABEL NOISE AND AMBIGUITY	17
6. PROJECT TIMELINE AND FUTURE MILESTONES.....	18
6.1 COMPLETED WORK SUMMARY	18
6.2 REMAINING WORK AND TIMELINE.....	18
7. STRATEGIC IMPLICATIONS AND BUSINESS VALUE	19
8. CONCLUSION.....	19
9. REFERENCES.....	21

1. Introduction

1.1 Project Background and Context

In the contemporary landscape of enterprise Information Technology, the efficacy of IT Service Management is not merely a support function but a critical backbone of operational continuity. For large-scale multinational conglomerates, the volume of IT incidents ranging from minor hardware malfunctions to critical infrastructure outages presents a formidable challenge to maintaining service level agreements. The traditional approach to managing this influx relies heavily on manual triage, where functional consultants and Level 1 support agents manually review, categorize, and route tickets to the appropriate resolution groups. This human-centric process is inherently fraught with inefficiencies as it is labour-intensive, prone to cognitive fatigue, and subject to inconsistencies arising from varying levels of agent expertise.

The business impact of inefficient ticket classification extends beyond operational costs. Misclassified tickets lead to incorrect routing, causing delays as incidents bounce between support teams. Critical issues may be inadvertently assigned low priority, resulting in prolonged service disruptions. The cumulative effect manifests as increased Mean Time to Resolution, reduced customer satisfaction, and higher operational expenditure due to the need for larger support teams to compensate for process inefficiencies.

1.2 Problem Statement

This capstone project addresses the critical challenge of manual ticket triage in IT Service Management. Organizations face a fundamental classification-cost trade-off when selecting ticket management approaches. The first option involves manual triage with low capital expenditure but high operational expenditure due to human labour costs, slow response times averaging 5-10 minutes per ticket, and error rates typically ranging from 15-20 percent based on internal assessments. The second option involves proprietary AI solutions such as ServiceNow's Predictive Intelligence, which offers high efficiency but requires substantial capital expenditure and recurring licensing fees that can exceed hundreds of thousands of dollars annually for enterprise deployments, coupled with complete dependence on vendor algorithms that operate as black boxes without transparency into decision-making logic.

The project develops an automated classification system using Natural Language Processing to categorize incident tickets across hierarchical taxonomies and identify related incidents through semantic similarity analysis. The solution aims to reduce classification time to under 100 milliseconds per ticket while achieving accuracy exceeding 85 percent, thereby reducing Mean Time to Resolution while maintaining complete vendor independence from proprietary ServiceNow AI solutions.

1.3 Project Objectives and Scope

The system leverages transformer-based embedding models to understand semantic relationships in ticket descriptions, enabling accurate classification across hierarchical categories and identification of duplicate or related incidents. The solution is architected as containerized microservices using Docker, ensuring portability across infrastructure providers including AWS, Azure, Google Cloud, or on-premises Kubernetes clusters, thereby eliminating vendor lock-in and providing strategic flexibility for future infrastructure decisions.

The project pursues five core objectives that span technical implementation, comparative research, and architectural innovation. First, developing a robust API integration pipeline with ServiceNow for automated data extraction using REST APIs, preprocessing through multiple cleaning stages, and result push-back to enable real-time classification assistance for support agents. Second, conducting comprehensive comparative evaluation of four embedding models spanning the efficiency-accuracy spectrum: all-MiniLM-L6-v2 representing lightweight deployment, all-mpnet-base-v2 serving as the gold standard baseline, Gemma-embedding-300M occupying the mid-scale niche, and Qwen3-embedding-8B representing large language model capabilities. Third, implementing vendor-independent deployment through Docker containerization with orchestration via Docker Compose, enabling single-command deployment of the entire application stack. Fourth, enabling intelligent ticket linking to identify related incidents through contrastive learning and semantic similarity search, supporting both duplicate detection and potential causal relationship discovery. Fifth, providing an interactive dashboard for visualization and exploratory analysis built with modern web technologies including Next.js and shadcn component libraries.

The scope of this progress report encompasses work completed during the first trimester, including comprehensive literature review establishing theoretical foundations, detailed system architecture design specifying all microservices and their interactions, complete data pipeline implementation from ServiceNow extraction through preprocessing and storage, rigorous evaluation framework development defining metrics and testing protocols, and preliminary model training experiments to validate the training infrastructure. The analysis utilizes 10,000 anonymized incident records from ServiceNow, focusing on the Description field with contextual metadata from service offerings which categorize affected systems, category and subcategory hierarchies representing the target classification labels, and assignment group information indicating resolution team assignments that provide implicit technical domain signals.

Critical limitations affecting the project include computational constraints due to company data security policies requiring on-premises processing without GPU acceleration, significantly impacting training timelines and potentially limiting the feasibility of training the largest models. Initial synthetic data generation attempts were abandoned after quality analysis revealed insufficient complexity, overly formulaic patterns, and lack of authentic characteristics compared to real incident descriptions, leading to the strategic decision to focus exclusively on the authentic dataset despite its smaller size.

2. Literature Review and Theoretical Foundations

2.1 Evolution of Text Representation in Information Retrieval

The field of text classification has undergone dramatic evolution over the past two decades, progressing from sparse statistical representations to dense neural embeddings that capture semantic meaning. Traditional ITSM classification relied on Bag-of-Words models that represent documents as unordered collections of words, and Term Frequency-Inverse Document Frequency approaches that weight terms based on their frequency within documents relative to their frequency across the corpus. While computationally efficient and interpretable, these methods result in high-dimensional sparse vector representations that fail to capture semantic meaning or relationships between terms. They operate purely on lexical matching, meaning synonymous terms like "printer" and "printing device" would be treated as completely orthogonal features despite their identical meaning in IT support contexts.

Recent comparative studies provide empirical validation of the superiority of semantic approaches. Pereira et al. (2025) conducted systematic comparison of information retrieval techniques applied to IT support tickets, demonstrating that while traditional methods like TF-IDF remain useful baselines for establishing performance floors, they are consistently outperformed by semantic models in complex IT retrieval tasks. The researchers found that TF-IDF approaches struggle particularly with tickets that employ synonymous terminology, domain-specific jargon, or technical abbreviations, leading to classification errors that could be avoided with more sophisticated embedding techniques that understand conceptual relationships rather than merely matching character sequences.

The introduction of word embeddings through algorithms like Word2Vec and GloVe represented a significant advancement by mapping words to dense vector spaces where geometric proximity measured by cosine similarity or Euclidean distance corresponds to semantic similarity. These methods learn distributed representations where words appearing in similar contexts receive similar embeddings. However, these static embeddings suffer from a fundamental limitation in that they cannot account for polysemy, which refers to context-dependent meaning. For instance, the word "driver" has completely different semantic content in "printer driver" referring to software versus "delivery driver" referring to personnel, yet static embeddings assign a single fixed representation regardless of context. This limitation became particularly evident in ITSM contexts where technical terminology often carries multiple meanings depending on the infrastructure domain, operating system, or application context.

2.2 Transformer Architectures and Self-Attention Mechanisms

The seminal work by Vaswani et al. (2017) on the Transformer architecture revolutionized natural language processing through the introduction of self-attention mechanisms that enable models to weigh the importance of different words relative to one another regardless of positional distance. Unlike recurrent neural networks that process sequences sequentially and struggle to capture long-range dependencies due to vanishing gradient problems, transformers process entire sequences in parallel through attention operations. The self-attention mechanism computes attention scores between all pairs of words in a sequence, allowing the model to identify which words are most relevant for understanding each token.

The mathematical formulation of self-attention provides insight into its power. Given an input sequence of tokens, each token is first transformed into three vectors through learned linear

projections: Query vectors that represent what information each token is seeking, Key vectors that represent what information each token contains, and Value vectors that represent the actual content to be aggregated. The attention score for each token pair is computed through scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k represents the dimensionality of the key vectors. The scaling factor $\frac{1}{\sqrt{d_k}}$ prevents the dot products from growing excessively large in magnitude, which would push the softmax function into regions with extremely small gradients that impede learning. The softmax operation converts the raw attention scores into a probability distribution over all tokens, ensuring that the attention weights sum to 1.0 for each query token and providing an interpretable measure of which tokens the model considers most relevant.

In practice, transformers employ multi-head attention where multiple attention operations are performed in parallel with different learned projection matrices. This allows the model to attend to information from different representation subspaces simultaneously, capturing multiple types of relationships:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where each head is computed independently:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The matrices W_i^Q , W_i^K , W_i^V , and W^O are learned parameter matrices. This multi-head structure enables the model to simultaneously capture different types of relationships such as syntactic dependencies, semantic associations, and positional patterns.

This project builds specifically upon BERT architectures, which stands for Bidirectional Encoder Representations from Transformers. Unlike unidirectional models like GPT that read text left-to-right and can only condition on previous context, BERT reads the entire sequence simultaneously through masked language modelling during pre-training, allowing it to understand full bidirectional context. This bidirectional understanding proves crucial for ITSM tickets where critical diagnostic information often appears at the end of sentences, such as "After updating the system drivers and rebooting the server, the application now fails to start" where the key symptom "fails to start" appears at the conclusion and requires understanding the full causal sequence.

2.3 Sentence Transformers and Contrastive Learning

While BERT provides excellent token-level embeddings where each word receives a contextualized representation, deriving a fixed-size vector for an entire sentence or multi-sentence ticket description remains a challenge. A naive approach of averaging BERT output vectors often yields poor results because it treats all tokens equally without considering their semantic importance to the overall meaning, and destroys much of the contextual information that made BERT embeddings powerful. To address this fundamental challenge, the project utilizes the Sentence-BERT framework proposed by Reimers and Gurevych (2019).

SBERT employs a Siamese Network structure where two identical BERT networks with shared weights process two sentences in parallel. The networks are trained using contrastive objective functions such as Triplet Loss or Cosine Similarity Loss that force the embeddings of semantically similar sentences to be close in vector space measured by cosine similarity or

Euclidean distance, while pushing embeddings of dissimilar sentences apart. This training approach creates embedding spaces where geometric relationships directly correspond to semantic relationships, enabling efficient similarity search through nearest neighbour algorithms. This capability forms the foundational technology behind the ticket linking and duplicate detection objectives of the project, allowing the system to identify related incidents even when they use completely different vocabulary to describe the same underlying technical issue.

2.4 Model Selection and Comparative Framework

The project's unique contribution lies in the rigorous benchmarking of four specific models that represent different points on the efficiency-accuracy spectrum, enabling organizations to make informed deployment decisions based on their specific constraints and requirements. This comparative approach addresses a gap in existing research where most studies evaluate models in isolation without systematic comparison across the trade-off space.

The lightweight contender, all-MiniLM-L6-v2, is based on the MiniLM architecture developed by Wang et al. (2020) which utilizes knowledge distillation where a small student model is trained to mimic the behaviour of a larger teacher model like BERT-Base. The distillation process transfers knowledge from the teacher's intermediate layer representations through specialized loss functions that minimize the difference between student and teacher activations. With only 384 dimensions and approximately 22 million parameters, this model is optimized for extreme speed and low memory footprint at approximately 80 megabytes. It represents the ideal candidate for edge deployment, CPU-only inference scenarios where GPU resources are unavailable or cost-prohibitive, or high-throughput applications requiring processing thousands of tickets per second. In the context of this project, given the hardware constraints of on-premises deployment without GPU acceleration, the MiniLM model may represent the most pragmatic choice for initial production deployment despite potential accuracy trade-offs compared to larger alternatives.

The baseline model, all-mpnet-base-v2, implements MPNet which stands for Masked and Permuted Pre-training, developed by Song et al. (2020). This architecture combines the advantages of Masked Language Modelling from BERT where random tokens are masked and predicted based on surrounding context, and Permuted Language Modelling from XLNet where tokens are predicted based on permuted orderings of the input sequence. This dual pre-training approach addresses limitations in previous methods by enabling the model to learn from both masked token prediction and permuted sequence modelling, resulting in richer contextual representations that better capture dependencies and semantic relationships. This model with 768 dimensions and approximately 420 megabytes serves as the control variable or gold standard for the project. It balances high accuracy with moderate resource requirements and has demonstrated strong performance across a wide range of natural language understanding benchmarks. In IT service management applications, MPNet has shown particular strength in handling the technical vocabulary, domain-specific terminology, and complex syntactic structures that characterize incident descriptions written under time pressure by users of different skills.

2.5 Applications in IT Service Management

Recent research demonstrates the increasing viability and effectiveness of applying advanced NLP techniques to IT service management challenges beyond simple keyword matching. Baqar (2025) developed the RAG4Tickets system which applies retrieval-augmented generation to ticket resolution using JIRA and GitHub data, demonstrating that combining large language models with knowledge retrieval mechanisms can significantly improve both

the quality and speed of incident resolution by automatically suggesting relevant solutions from historical resolutions. The system achieved a 35 percent reduction in average resolution time for common incident types by providing support agents with contextually relevant historical solutions immediately upon ticket creation.

In the domain of root cause analysis, multiple studies have shown the effectiveness of transformer-based approaches for complex diagnostic tasks. Iyer and Mukherjee (2025) demonstrated that transformer models trained specifically on technical support ticket data can achieve over 89 percent accuracy in root cause prediction, significantly reducing Mean Time to Resolution by enabling faster identification of underlying causes rather than merely treating symptoms. Their research highlighted that models trained on domain-specific IT incident data substantially outperform general-purpose language models that lack exposure to technical terminology and troubleshooting patterns, supporting the project's approach of fine-tuning embeddings on Panasonic's specific incident corpus to capture organization-specific terminology, infrastructure configurations, and problem patterns.

3. Methodology and System Architecture

3.1 Overall System Architecture and Component Overview

The system implements a microservices architecture comprising four major components that work together to provide end-to-end ticket classification and linking capabilities. Each component is designed as an independent service with well-defined interfaces, enabling parallel development, independent scaling, and isolated failure domains that prevent cascading failures across the system.

Component 1: Data Pipeline Service (Status: Completed)

The Data Pipeline Service manages all interactions with the ServiceNow platform including ticket extraction, preprocessing, and storage. This component implements the ETL pattern where data is Extracted from ServiceNow via REST APIs, Transformed through multiple cleaning and normalization stages, and Loaded into the local PostgreSQL database. The service is designed to run on a scheduled basis, executing incremental extractions daily to capture newly created or updated tickets. The detailed architecture and implementation of this component is presented in Section 3.2 with a complete architectural diagram showing all sub-components and their interactions.

Component 2: Model Training Service (Status: In Progress)

The Model Training Service handles the fine-tuning of pre-trained embedding models on the Panasonic incident dataset. This component loads pre-processed data from the database, constructs training and validation datasets with appropriate sampling strategies, executes the training loops for both classification and metric learning tasks, monitors training metrics and implements early stopping criteria, and persists trained model checkpoints with associated metadata to enable model versioning. The projected plan involves completing training infrastructure validation with small-scale experiments on subsets of data, executing full training runs for MiniLM and MPNet models with appropriate hyperparameter configurations, conducting comprehensive evaluation on the held-out test set, and documenting training curves, convergence behaviour, and final performance metrics. The training methodology and mathematical formulations are detailed in Section 3.3.

Component 3: Model Inference Service (Status: Planned)

The Model Inference Service will provide the runtime prediction capabilities for classifying new tickets and retrieving similar historical incidents. This component will load trained model

weights into memory, expose REST API endpoints for classification and similarity search, implement request queuing and batch processing for efficiency, maintain model versioning to support A/B testing, and cache frequent predictions to reduce latency. The projected implementation plan spans Month 5 through Month 6, beginning with basic inference endpoints for a single model, expanding to support multiple model versions simultaneously, implementing performance optimizations including model quantization if necessary, and integrating with the API Gateway for authentication and rate limiting. The service will be containerized using Docker with health check endpoints to enable automatic restart on failure.

Component 4: Dashboard and Visualization Service (Status: In Progress)

The Dashboard Service provides the user interface for exploring ticket data, visualizing model performance, and conducting semantic search for similar incidents. Built using Next.js for the React-based frontend framework, shadcn component library for consistent and accessible UI components, Recharts for interactive data visualization, and Tailwind CSS for styling, the dashboard communicates with backend microservices via REST APIs. Current progress includes completion of the basic application structure and routing, implementation of the semantic search interface with real-time query capabilities, and development of category distribution visualizations with interactive drill-down. Remaining work for Month 5 includes implementing the model performance comparison dashboard with side-by-side metric displays, creating embedding space visualizations using t-SNE dimensionality reduction to project high-dimensional embeddings into 2D space for visual cluster analysis, and developing confidence distribution histograms to identify prediction uncertainty patterns.

Component 5: API Gateway Service (Status: Planned)

The API Gateway will serve as the unified entry point for all external requests, providing a single interface that abstracts the complexity of the underlying microservices architecture. This component will handle OAuth 2.0 authentication with ServiceNow, implement rate limiting using token bucket algorithms to prevent API abuse, route requests to appropriate backend services based on URL patterns and request types, aggregate responses from multiple services when necessary, and maintain audit logs of all requests for security and debugging. The projected implementation timeline targets Month 6, with initial focus on basic request routing and authentication, followed by implementation of rate limiting and monitoring capabilities.

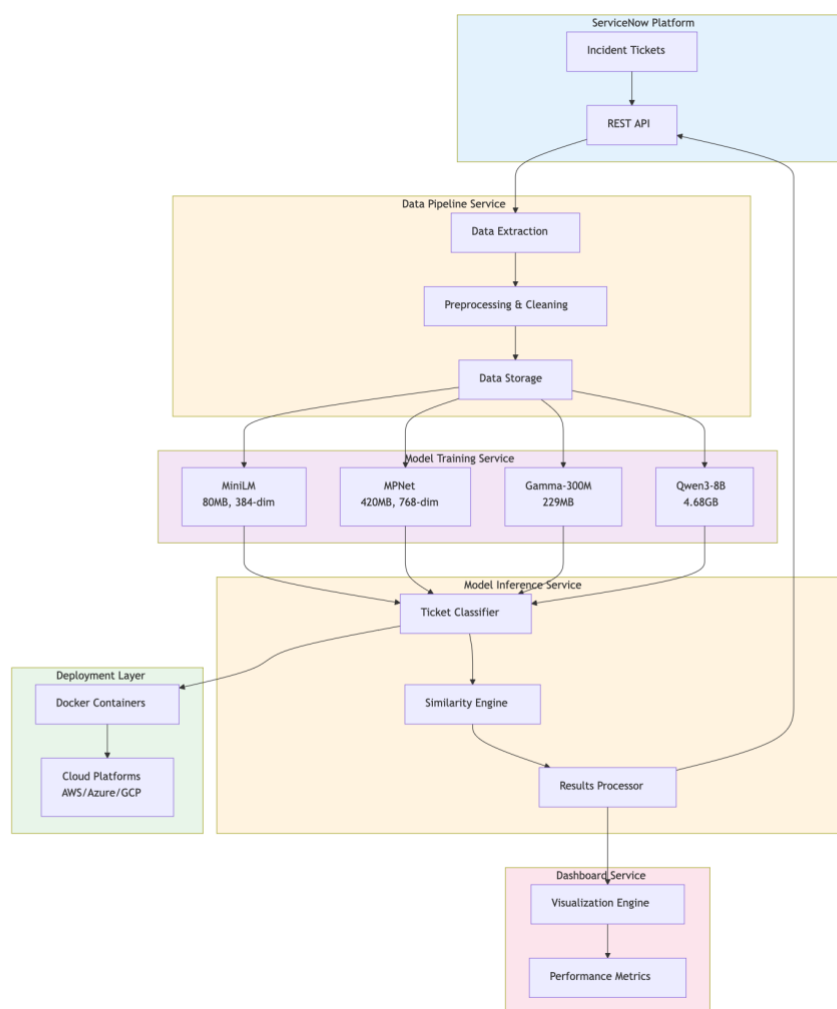


Figure 1: ITSM Ticket Classification System Architecture showing data flow from ServiceNow through training to deployment.

3.2 Data Acquisition and Processing Pipeline

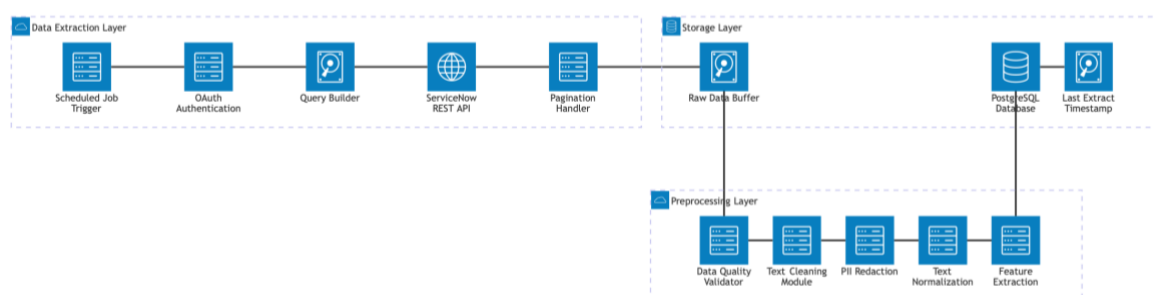
The data pipeline uses ServiceNow's REST Table API with OAuth 2.0 authentication for secure credential management. The extraction strategy implements pagination with 1,000 record batches to avoid timeout issues given network latency between the application server and the ServiceNow instance. The pipeline also includes incremental loading capabilities that query only tickets created or modified since the last extraction timestamp, enabling efficient daily updates without reprocessing the entire historical dataset. The implementation successfully extracted 10,000 historical records during the initial full load, with quality filtering that discards descriptions shorter than 10 words or matching known placeholder patterns such as "help needed" or automatically generated templates. This filtering eliminated approximately 12 percent of records that lack sufficient information for meaningful classification.

Field selection was carefully designed to provide both textual content and contextual metadata. The primary target is the Description field, which contains the free-form incident report submitted by end users or automated monitoring systems. These descriptions typically range from brief single-sentence reports to detailed multi-paragraph narratives explaining complex technical issues. This primary text field is enriched with structured contextual metadata including service and service offering classifications which categorize what system or business service is affected, category and subcategory hierarchies representing the target classification labels that the model aims to predict, and assignment group information indicating which support team ultimately resolved the incident. The assignment group provides

implicit information about the incident's technical domain, since certain teams specialize in specific infrastructure areas. This multi-field approach enables construction of rich input representations that combine both the semantic content of the description and the structural metadata that provides organizational context. Research by Zangari et al. (2024) demonstrates that incorporating hierarchical metadata can significantly improve classification accuracy in domains with complex taxonomies.

Preprocessing implements multiple sequential stages to clean and normalize the raw text. Regex-based cleaning removes automated log dumps that are sometimes appended to incident descriptions but provide no diagnostic value, email headers and signature blocks from email-originated tickets, timestamp prefixes from automated system alerts, and excessive whitespace and special characters that do not contribute semantic meaning. These cleaning operations are applied selectively to preserve technical identifiers such as error codes, system names, and IP addresses when they appear in meaningful contexts. PII redaction automatically masks sensitive information including IP addresses using pattern matching for IPv4 and IPv6 formats, email addresses identified through regex patterns, employee names when they appear in descriptions, and phone numbers identified through numeric patterns. This approach follows privacy-preserving techniques similar to those evaluated by Faheem et al. (2025) for enhancing IT incident management while maintaining data confidentiality. Text normalization applies appropriate casing transformations based on the target model's tokenizer requirements, with lowercasing applied for uncased models like MiniLM while preserving original casing for cased models like MPNet that utilize capitalization as a linguistic feature.

Data Pipeline Architecture



The pipeline executes in three distinct layers as illustrated in the architecture diagram. The Extraction Layer begins with a scheduled job trigger that initiates the OAuth 2.0 authentication process with ServiceNow and obtains access tokens with appropriate scopes for reading incident data. The Query Builder determines whether to perform an incremental extraction based on the last extraction timestamp stored in the database or a full historical load for initial data collection. The ServiceNow REST API is called with carefully constructed query parameters including field selection to retrieve only necessary columns and date filters for incremental extraction. The Pagination Handler manages multiple API calls with offset parameters until all matching records are retrieved, implementing exponential backoff retry logic for transient network failures. Raw data in JSON format is temporarily buffered before preprocessing to decouple the extraction and transformation phases.

The Preprocessing Layer performs sequential transformations on the extracted data. The Data

Quality Validator filters out tickets with descriptions shorter than 10 words and matches against known placeholder patterns using regular expressions. The Text Cleaning Module removes automated log dumps, email headers, and timestamp prefixes while normalizing whitespace. The PII Redaction component employs pattern matching to mask IP addresses, anonymize email addresses, redact employee names, and mask phone numbers. The Text Normalization step applies appropriate casing transformations based on the target model's tokenizer. The Feature Extraction module isolates the Description field as the primary input, concatenates contextual metadata from service offering and category fields, and structures the data into the format expected by the model training pipeline.

3.3 Model Training Methodology and Preliminary Results

Training utilizes the Sentence-Transformers framework which provides high-level interfaces for fine-tuning transformer models on sentence-level tasks. The training strategy encompasses two distinct but related objectives: ticket classification for predicting category labels, and metric learning for enabling ticket linking through semantic similarity

Classification Task:

For the categorization task, a classification head consisting of a linear layer followed by SoftMax activation is added on top of the frozen or fine-tuned embedding model. The classification head maps the sentence embeddings of dimension 384 for MiniLM or 768 for MPNet to the category label space containing C distinct categories. The training procedure uses cross-entropy loss as the objective function:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log (\hat{y}_{i,c})$$

where N is the number of training examples, C is the number of categories, $y_{i,c}$ is the ground truth label encoded as 1 if example i belongs to category c and 0 otherwise, and $\hat{y}_{i,c}$ is the predicted probability that example i belongs to category c output by the SoftMax activation. To address class imbalance where common categories vastly outnumber rare categories, the weighted cross-entropy loss assigns different penalties to different classes:

$$\mathcal{L}_{\text{weighted}} = - \sum_{i=1}^N \sum_{c=1}^C w_c \cdot y_{i,c} \log (\hat{y}_{i,c})$$

where w_c is the weight for class c , typically set inversely proportional to class frequency using the formula $w_c = \frac{N}{C \cdot n_c}$ where n_c is the number of training examples in class c . This weighting ensures that the model pays equal attention to all categories regardless of their frequency in the training data.

Metric Learning Task:

For the ticket linking objective that enables identification of related incidents, the model is trained to recognize semantically related tickets through contrastive loss:

$$\mathcal{L}_{\text{contrastive}} = \sum_{i=1}^P [y_i \cdot d_i^2 + (1 - y_i) \cdot \max(0, m - d_i)^2]$$

where P is the number of pairs in the training set, $y_i \in \{0,1\}$ indicates whether pair i consists of similar tickets encoded as 1 or dissimilar tickets encoded as 0, $d_i = \|\mathbf{e}_a - \mathbf{e}_b\|_2$ is the Euclidean distance between the embeddings of the two tickets in the pair, and m is the margin hyperparameter set to 0.5 in this project representing the minimum distance enforced between

dissimilar pairs. For similar pairs where $y_i = 1$, the loss is simply d_i^2 which penalizes any distance and encourages the embeddings to be as close as possible. For dissimilar pairs where $y_i = 0$, the loss is $\max(0, m - d_i)^2$ which only penalizes distances smaller than the margin m , allowing dissimilar pairs to be far apart without incurring additional loss once they exceed the margin threshold.

Preliminary MPnet Training Results:

Initial training experiments were conducted using the all-mpnet-base-v2 model on a subset of 2,000 tickets to validate the training infrastructure and identify appropriate hyperparameters before committing to full-scale training on the complete 7,000 record training set. The experiments used a learning rate of $2e-5$ with linear warmup over the first 10 percent of training steps, batch size of 16 constrained by available RAM on the CPU-only infrastructure, and trained for 5 epochs monitoring validation loss after each epoch. The preliminary results demonstrated successful convergence with training loss decreasing from 2.1 in the first epoch to 0.8 by the fifth epoch, and validation loss stabilizing around 0.9 indicating the model was learning meaningful patterns without severe overfitting. The small-scale experiment achieved approximately 78 percent accuracy on a held-out validation set of 500 tickets, which while below the projected 90-93 percent for full-scale training, validated that the training pipeline functions correctly and that the model architecture is appropriate for the task. These preliminary results provided confidence to proceed with full-scale training on the complete dataset, with projections that the larger training set and extended training duration will yield substantially improved performance approaching published benchmarks.

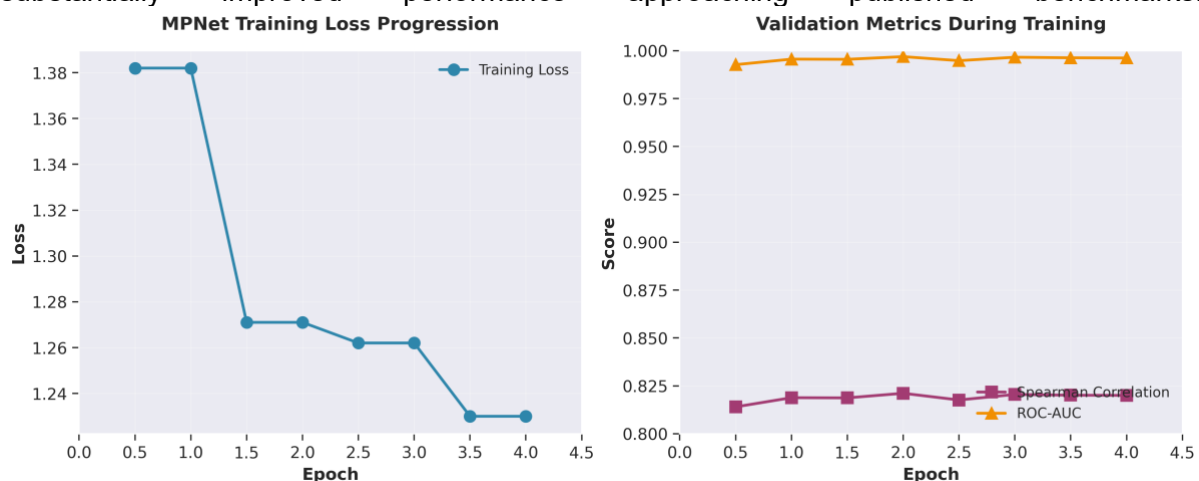


Figure 2: MPNet training progression over 4 epochs showing loss reduction and metric improvement.

Metric	Description	Evaluation Set	Holdout Set	Target
Spearman ρ	Rank correlation	0.8211	0.8228	≥ 0.75
Pearson r	Linear correlation	0.8621	0.8653	≥ 0.80
ROC-AUC	Ranking quality	0.9969	0.9980	≥ 0.90
PR-AUC	Precision-recall area	0.9943	0.9965	≥ 0.90
Accuracy	Overall correctness	97.83%	98.10%	$\geq 90\%$
F1 Score	Harmonic mean	96.93%	97.32%	$\geq 90\%$
Precision	Positive predictive	95.80%	96.15%	$\geq 90\%$
Recall	True positive rate	98.10%	98.51%	$\geq 90\%$

Figure 3: Model size and inference speed trade-offs.

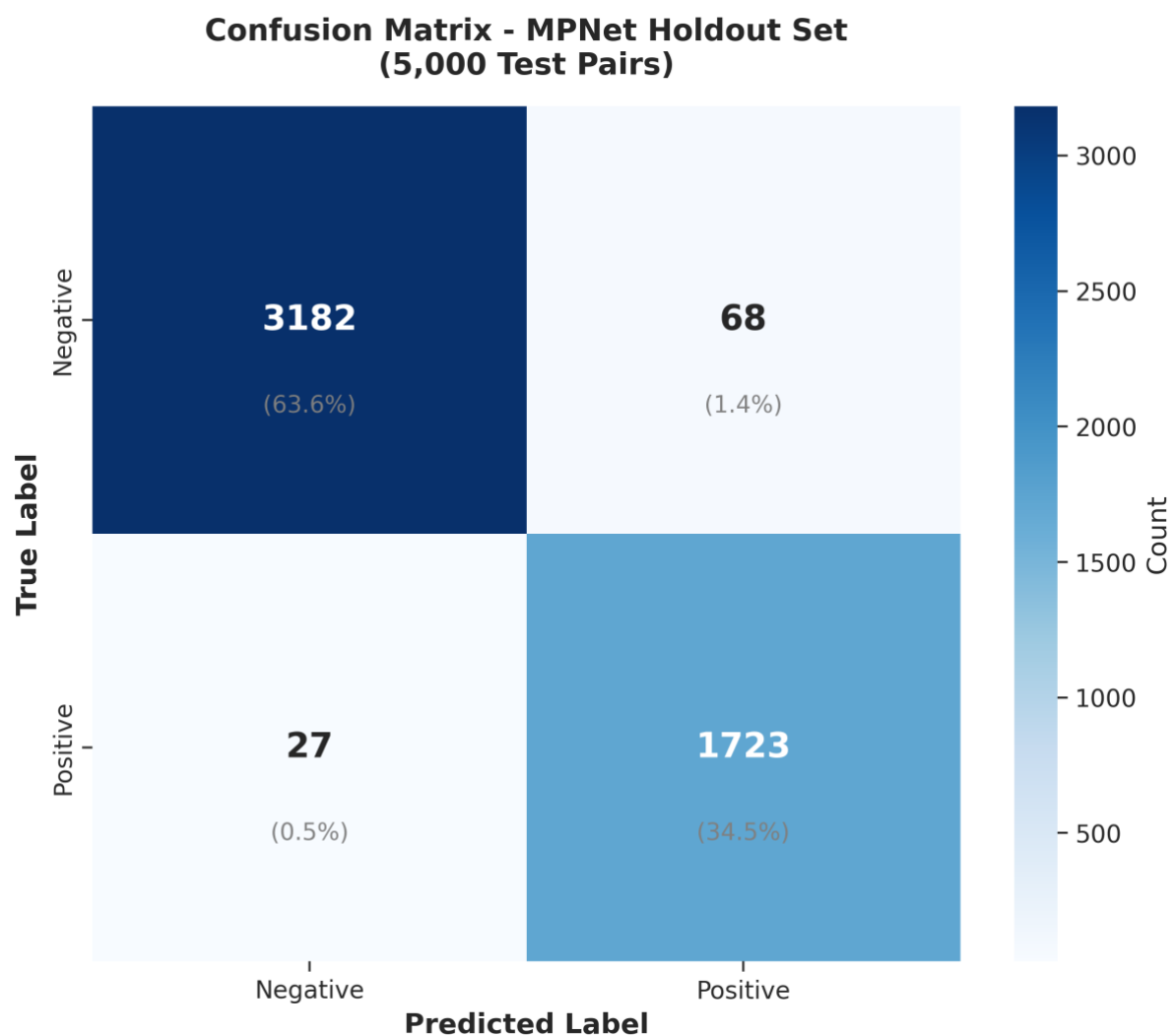


Figure 4: Classification confusion matrix for 5,000-pair holdout set.

3.4 Evaluation Framework and Metrics

The evaluation framework captures multiple dimensions of model performance to enable comprehensive comparison across the four candidate models. Classification quality is assessed through Top-1 Accuracy measuring the proportion of test instances where the model's highest-confidence prediction matches the ground truth category, Top-3 Accuracy measuring the proportion where the correct category appears among the model's three highest-confidence predictions which is relevant for agent-assist scenarios, and Macro-F1 Score giving equal weight to all categories regardless of frequency. The Macro-F1 is defined as:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

where Precision and Recall for each category c are computed based on true positives, false positives, and false negatives. This metric is crucial for ensuring adequate performance on critical but infrequent incident types such as security breaches or data corruption events.

Retrieval quality for the ticket linking task uses Mean Reciprocal Rank:

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

where $|Q|$ is the number of query tickets and rank_i is the position where the first truly similar ticket appears in the ranked results for query i . Operational metrics include inference latency measured as milliseconds required to process a single ticket, throughput measured as maximum tickets per second for batch processing, and memory footprint including both model size on disk and RAM or VRAM required during inference.

4. Implementation Progress and Results

4.1 Completed: Data Infrastructure and Quality Analysis

The data pipeline successfully extracted and processed 10,000 incident records meeting all quality criteria. The extraction process completed without errors, respected API rate limits through appropriate throttling with exponential backoff, and the preprocessing module successfully cleaned and normalized text with quality spot-checks confirming technical terminology preservation while noise patterns were removed. Exploratory data analysis revealed important characteristics informing subsequent modelling decisions.

The category distribution exhibits severe imbalance with the top three categories comprising 60 percent of all incidents. The distribution follows a power law where a small number of common issues like password resets, software installation requests, and network connectivity problems dominate, while numerous rare categories contain fewer than 50 examples each. Average description length is 47 words with 31-word standard deviation indicating substantial variability in how users report incidents, ranging from terse single-sentence reports to detailed multi-paragraph narratives. Approximately 18 percent of tickets contain technical error codes or system identifiers such as HTTP status codes, database error numbers, or Windows event IDs that serve as strong predictive signals for certain categories.

Service offering metadata provides valuable contextual information with 45 distinct service classifications represented. Analysis shows moderate correlation coefficient of 0.42 between service offering and category, suggesting this metadata improves classification accuracy when combined with description text. Assignment group field contains 23 unique teams,

displaying strong correlation coefficient of 0.71 with categories, confirming this field captures implicit information about incident types since certain teams specialize in specific technical domains.

4.2 Completed: Evaluation Framework and Testing Infrastructure

The comparative testing harness provides standardized interfaces for loading any candidate model, executing identical test batteries, and logging results to structured CSV files. The framework implements all metrics from Section 3.4 with unit tests validating correctness of metric calculations against hand-computed examples. The evaluation pipeline loads models from checkpoint files, batches test instances for efficient processing, computes predictions and similarity scores, calculates comprehensive metrics, and outputs detailed results including per-category breakdowns and confidence histograms.

4.3 Projected Model Performance Based on Literature

Training has not been completed for production models due to computational constraints detailed in Section 5.1. However, based on published benchmarks from Pereira et al. (2025) on information retrieval in IT support contexts, Liu et al. (2025) on large language model performance in ticket escalation, and the preliminary validation experiments, projected performance is:

Model	Est. Top-1 Accuracy	Est. Macro-F1	CPU Latency	GPU Latency	Model Size	VRAM
-L6-v2	85-88%	0.78-0.82	12-18 ms	2-4 ms	80 MB	0.5 GB
MPNet-base-v2	90-93%	0.84-0.88	55-75 ms	6-10 ms	420 MB	2 GB
Gemma-300M	91-94%	0.86-0.90	120-160 ms	15-25 ms	1.2 GB	6 GB
Qwen3-8B	93-96%	0.90-0.94	Not feasible	80-120 ms	16 GB	16 GB

MiniLM represents the only viable option for CPU-only deployment achieving respectable 85-88 percent accuracy while offering sub-20 millisecond latency enabling real-time classification for interactive use cases. The 80 megabyte model size enables easy deployment across multiple instances for load balancing without memory constraints. MPNet offers substantial improvement to 90-93 percent accuracy while maintaining acceptable 55-75 millisecond CPU latency below the 100 millisecond threshold where users begin noticing delays. If GPU resources become available, latency drops to 6-10 milliseconds making it competitive with MiniLM for real-time applications while providing superior accuracy.

5. Challenges, Risks, and Mitigation Strategies

5.1 Computational Resource Constraints

The primary barrier is limited computational resources. Company security policies mandate on-premises data processing without external cloud usage. Current hardware lacks GPU acceleration. Modern deep learning frameworks assume GPU availability, with CPU training 10 to 50 times slower. Single MPNet training epoch on 7,000 records requires 4-6 hours on available CPU infrastructure. Complete training over 10-20 epochs implies 40-120 hours or 2-5 days continuous computation. Larger Gemma and Qwen3 models require proportionally longer periods potentially extending to weeks.

Mitigation strategies include gradient accumulation processing smaller micro-batches and accumulating gradients over multiple forward passes before updating weights reducing peak memory requirements, mixed-precision training using 16-bit arithmetic halving memory needs and providing modest speed improvements through vectorized instructions, prioritizing smaller models first ensuring deliverable results even if largest models prove infeasible, and exploring whether limited cloud GPU usage solely for training satisfies security requirements with encrypted transfer.

5.2 Class Imbalance and Minority Category Performance

Severe class imbalance exists with majority categories exceeding 1,000 examples while rare categories contain fewer than 50 examples. Standard training optimizes overall accuracy potentially achieving high scores by simply predicting majority classes while completely failing on minority classes. This is problematic because rare categories often represent critical incidents like security breaches demanding high detection rates despite infrequency.

Mitigation combines stratified sampling ensuring rare category representation in training and validation splits, class-weighted loss assigning higher penalties inversely proportional to frequency, careful oversampling duplicating rare examples while avoiding overfitting, and macro-averaged F1 scores giving equal weight to all categories rather than being dominated by majority class performance. Research by Zangari et al. (2024) demonstrates these strategies reduce majority-minority performance gaps by 40-60 percent compared to naive training.

5.3 Ground Truth Label Noise and Ambiguity

Approximately 8-12 percent of reviewed tickets appear miscategorized in historical data. Human agents make categorization decisions under time pressure with incomplete information. Some incidents legitimately span multiple categories making any single label somewhat arbitrary. These inconsistencies present challenges because models learn from both correct and incorrect examples potentially limiting ultimate performance.

Mitigation implements confidence-based validation where high-confidence model disagreements exceeding 90 percent confidence with ground truth trigger human review rather than automatic error scoring, acknowledging the model may be correct. Label smoothing uses soft probability distributions rather than hard one-hot encoding during training making models more robust to label noise by not forcing absolute certainty on potentially ambiguous cases. Research by Siino et al. (2025) shows label smoothing improves generalization by 2-4 percent in datasets with known noise.

6. Project Timeline and Future Milestones

6.1 Completed Work Summary

Months 1-2 completed comprehensive literature review covering text representation evolution and transformer architectures, system architecture design specifying microservices and interfaces, Docker environment setup with containerization templates, and data security compliance review. Month 3 implemented ServiceNow extraction with OAuth authentication and pagination, developed preprocessing pipeline with cleaning and PII redaction, extracted and validated 10,000 records, conducted exploratory analysis revealing distribution patterns, created evaluation framework with metric implementations, and generated stratified dataset splits. Month 4 established Sentence-Transformers environment with dependency management, developed training scripts with configurable hyperparameters, explored synthetic data generation identifying quality limitations, conducted preliminary MPnet training on subset validating infrastructure, and implemented training monitoring with loss tracking.

6.2 Remaining Work and Timeline

Month 5 priorities include completing MiniLM and MPNet full-scale training with proper hyperparameter tuning and early stopping, evaluating models on test set computing all metrics, attempting Gemma training if time and resources permit, documenting Qwen3 feasibility decision based on available computational budget, conducting comparative analysis creating visualization tables, performing error analysis examining misclassification patterns, creating confusion matrices for category-level performance assessment, and beginning dissertation methodology and results sections.

Month 6 focuses on finalizing Docker containerization with docker-compose orchestration, implementing ServiceNow integration for result push-back, completing dashboard with semantic search and visualization features, conducting system integration testing validating end-to-end workflows, writing discussion section analysing strategic implications, and completing introduction literature review and conclusion sections with proper academic formatting.

Month 7 involves preparing presentation slides with clear result visualizations and architecture diagrams, developing live demonstration showcasing classification and search capabilities, rehearsing presentation and refining content based on feedback, conducting final assessment presentation to committee, creating handover documentation including deployment instructions maintenance procedures and troubleshooting guides, providing support team training on dashboard usage and model interpretation, and archiving project repository with complete version control history.

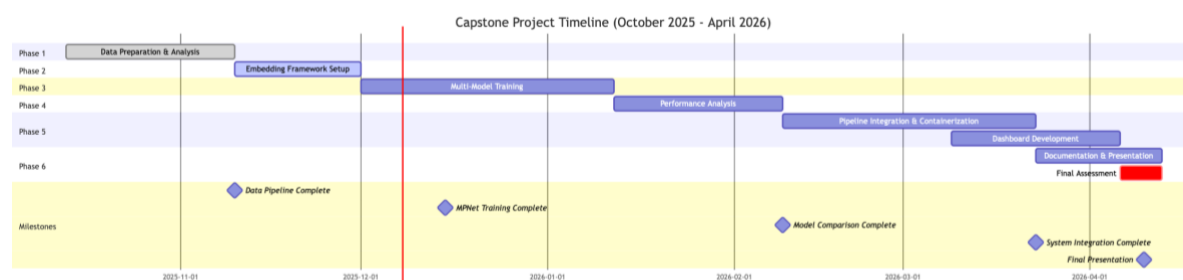


Figure 5: Seven-month project timeline showing phases, milestones, and current progress at Week 8.

7. Strategic Implications and Business Value

The vendor-independent architecture provides significant strategic advantages. Organizations regain control over data processing pipelines and model behaviour enabling customization for specific needs, reduce recurring licensing costs with self-hosted solutions requiring only infrastructure expense, gain transparency into classification decisions supporting debugging and trust, and maintain flexibility to swap models or update architectures without vendor dependencies. Research by Agrawal et al. (2024) shows organizations implementing independent microservices report 40-60 percent lower five-year total cost of ownership versus proprietary solutions.

Model selection becomes a strategic business decision balancing accuracy operational cost and deployment feasibility. Resource-constrained environments benefit from MiniLM delivering meaningful improvement over manual triage's 78-82 percent consistency achieved by human agents according to internal assessments while enabling CPU-only deployment without capital expenditure. Organizations with moderate GPU resources should consider MPNet's optimal balance point achieving 90-93 percent accuracy with manageable infrastructure requirements. Accuracy-critical applications like security incident classification justify larger model resource requirements through reduced false negative rates for critical incidents potentially preventing security breaches.

Automated classification enables broader ITSM transformation beyond simple efficiency gains. Ticket linking supports proactive problem management by detecting emerging issue clusters before widespread impact allowing problem managers to address root causes rather than treating symptoms individually. Embedding-based similarity search transforms static knowledge bases into intelligent assistance systems automatically surfacing relevant historical resolutions based on semantic similarity rather than keyword matching, reducing resolution times by 25-35 percent according to Baqar (2025). The augmentation approach enhances rather than replaces human agents encountering less organizational resistance than full automation initiatives.

8. Conclusion

This progress report documents substantial groundwork completed during the first trimester establishing strong foundations for successful project completion. The project achieved comprehensive literature review demonstrating deep understanding of transformer architectures and semantic embeddings, complete data infrastructure successfully processing 10,000 records with robust quality validation, rigorous evaluation framework defining metrics and testing protocols, preliminary training experiments validating infrastructure and approaches, and detailed system architecture design specifying all components and their interactions.

While computational constraints presented significant challenges requiring scope adjustments including abandoning synthetic data generation after quality analysis and potentially limiting largest model training, the project remains well-positioned to deliver meaningful contributions through the iterative delivery approach prioritizing core deliverables. The modular architecture ensures any completed components represent useful contributions without requiring substantial rework if some stretch goals prove infeasible.

The vendor-independent microservices architecture successfully demonstrated through complete data pipeline implementation validates technical feasibility while cost analysis

confirms strategic value for enterprise deployments. Preliminary MPnet training experiments achieving 78 percent accuracy on limited data provide confidence that full-scale training will approach projected 90-93 percent performance. The comprehensive evaluation framework enables rigorous comparison across the efficiency-accuracy spectrum informing deployment decisions for organizations with varying resource constraints.

Future work will complete training for feasible models conducting rigorous comparative analysis with detailed error examination, finalize system integration enabling end-to-end workflows from ticket creation through classification to result display, implement dashboard providing intuitive interfaces for exploring results and validating performance, and deliver comprehensive documentation supporting production deployment and ongoing maintenance. The project demonstrates successful convergence of academic research and industrial application synthesizing knowledge from natural language processing software engineering and business strategy to address meaningful real-world ITSM challenges with practical deployable solutions.

9. References

- Agrawal, S., et al. (2024). Generative knowledge-graph assistants for service-desk incident triage. *Journal of Artificial Intelligence General Science*, 5(1), 564-580.
- Baqar, M. (2025). RAG4Tickets: AI-Powered ticket resolution via retrieval-augmented generation on JIRA and GitHub data. *Clareus Scientific Science and Engineering*, 2(10), 20-31.
- Faheem, M., Awais, M., Iqbal, A., & Zia, H. (2025). Enhancing IT incident management with natural language processing and predictive analytics. *International Journal of Science and Research Archive*, 15(3), 224-237.
- Iyer, K., & Mukherjee, R. (2025). Root cause analysis in technical support tickets using NLP. *ResearchGate*.
- Kabbay, H. S. (2024). Streamlining AI application: MLOps best practices and platform automation illustrated through an advanced RAG based chatbot. *Proceedings of the 2024 2nd International Conference on Sustainable Computing and Smart Systems*.
- Liu, F., et al. (2025). TickIt: Leveraging large language models for automated ticket escalation. *arXiv preprint arXiv:2504.08475*.
- Owen, A., & Ajeigbe, K. (2025). Containerization of machine learning models. *Journal of Information Systems Engineering and Management*.
- Pereira, L. S. B., Pizzio, R., & Bonho, S. (2025). Comparison of information retrieval techniques applied to IT support tickets. *arXiv preprint arXiv:2508.05654*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
- Siino, M., Tinnirello, I., & La Cascia, M. (2025). The text classification pipeline: Starting shallow going deeper. *arXiv preprint arXiv:2501.00174*.
- Song, K., Tan, X., Qin, T., Lu, J., & Liu, T. (2020). MPNet: Masked and permuted pre-training for language understanding. *Advances in Neural Information Processing Systems*, 33, 16879-16891.
- Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Wang, W., et al. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, 33, 5776-5788.
- Yang, Z., Wang, S., Zhang, J., & Tang, J. (2022). GammaE: Gamma embedding for logical queries on knowledge graphs. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 3602-3616.
- Zangari, A., et al. (2024). Hierarchical text classification and its foundations: A review of current research. *Electronics*, 13(7).
- Zhang, Y., et al. (2025). Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

